

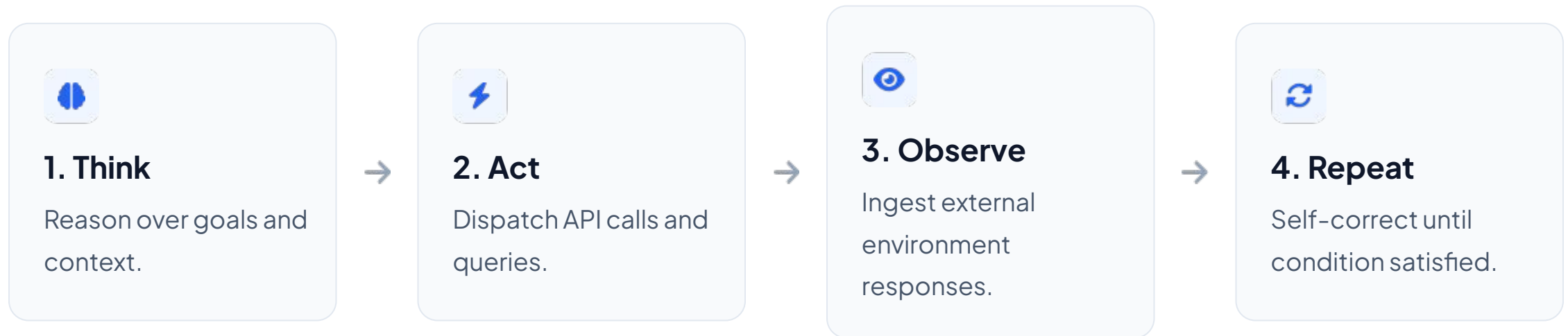
SYSTEM ARCHITECTURE & ORCHESTRATION

Agentic AI: From Raw Generation to Autonomous Systems

Closed-loop execution, cyclic state graphs, memory tiers, and production guardrails.

— LangGraph • Multi-Agent Systems • Evals

The Agentic Cycle: Think → Act → Observe



Standalone LLM vs. Agentic System

Capability	Standalone LLM	Agentic Architecture
Execution Loop	Single-turn prompt/response	Multi-turn closed-loop cycles
Environment Impact	Passive text emission	Active tool and API dispatch
Error Recovery	No self-verification	Dynamic retry on feedback
Decision Scope	Predicts next token	Determines operational sequence

Developer Control vs. Model Orchestration

Workflow as Code

Hardcoded branching with targeted model compute steps.

Determinism: Predictable paths and costs

Control Flow: Explicit state transitions

Best Fit: Regulated enterprise workflows

LLM as Orchestrator

Autonomous routing using tools and sub-agent delegates.

Adaptability: Dynamic error recovery

Planning: Emergent step breakdown

Best Fit: Open-ended research tasks

LangGraph Framework & Primitives

LangGraph is a framework for building agents as workflows. It connects steps, keeps track of data, and decides what happens next.

Part	Simple Meaning	OR Agent Example
State	What the agent knows	Scenario and solver result
Node	A step that does work	Run the optimizer
Edge	What happens next	Explain or diagnose
Memory	What it can recall	Earlier scenario results

STATE GRAPH IMPLEMENTATION

```
builder = StateGraph(AgentState)
builder.add_node("solve_model", solve_model)
builder.add_node("explain_result", explain_result)

builder.add_edge(START, "solve_model")
builder.add_edge("solve_model", "explain_result")
builder.add_edge("explain_result", END)

graph = builder.compile()
```

State: What the Agent Knows Now

State is the data shared between nodes. Each node reads it and returns an update.

IN OUR OR AGENT EXAMPLE

User Request

“What if Factory A loses 40% capacity?”

Scenario

Factory A capacity = 60%

Solver Result

Status = optimal; cost = ₹1.38 lakh

Key Point: State holds the current scenario and results.

PYTHON SCHEMA IMPLEMENTATION

```
class AgentState(TypedDict):  
    question: str  
    capacity: int  
    solver_status: str  
    cost: float
```

The `AgentState` structure dictates what data transitions through each graph node during execution.

Nodes: What the Agent Does

A node is a function that reads state, does work, and returns an update.

IN OUR OR AGENT EXAMPLE

`load_data`

Read capacities and demand

`solve_model`

Call the optimization solver

`explain_result`

Explain the solver output

Key Point: Nodes do the work. They can use Python, an LLM, or a tool.

PYTHON NODE IMPLEMENTATION

```
def solve_model(state):  
    result = solver(state["capacity"])  
  
    return {  
        "solver_status": result.status,  
        "cost": result.cost,  
    }
```

Node functions accept current `state` and return a dictionary update that mutates or appends to the state graph.

Memory: What the Agent Remembers

Memory makes information available beyond one step.

IN OUR OR AGENT EXAMPLE

Short-term Memory

Scope: Same conversation

Remember the baseline and current scenario

Long-term Memory

Scope: Across conversations

Remember a user's preferred comparison format

Key Point: State is current working data. Checkpointers handle short-term memory, stores handle long-term memory.

PYTHON MEMORY IMPLEMENTATION

```
graph = builder.compile(  
    checkpoint=checkpoint, # Short-term  
    store=store, # Long-term  
)
```

State is the current working data. A `checkpoint` saves it within a conversation; a `store` saves selected information across conversations.

Conditional Edges & Decision Gates



Tool Routing

Dynamic function dispatch based on state payload.



Schema Gates

Strict structural parameter verification.



Cycle Cutoffs

Hard iteration ceilings to stop loops.

Three Tiers of Agentic Memory

TIER 1

Working Memory

In-session scratchpad with sliding window pruning.

TIER 2

Semantic Store

Episodic vector databases across lifetime sessions.

TIER 3

Checkpoints

Serialized graph states enabling human replay.

Short-Term Memory vs. Long-Term Memory

CHECKPOINTER

Short-Term Memory

SCOPE

Single conversation thread (`thread_id`)

LANGGRAPH ABSTRACTION

Checkpoint (e.g., PostgresSaver, SqliteSaver)

PRIMARY JOB

Multi-turn dialog context, tool returns, pause/resume

LIFECYCLE

Ephemeral or bounded by session lifecycle

ACCESS PATTERN

Loaded automatically into node state

BASESTORE

Long-Term Memory

SCOPE

Cross-thread, global or user-scoped (`namespace`, `key`)

LANGGRAPH ABSTRACTION

BaseStore (e.g., InMemoryStore, DB vector stores)

PRIMARY JOB

Personalized preferences, user profiles, past learnings

LIFECYCLE

Indefinite; survives new threads and restarts

ACCESS PATTERN

Queried via key lookup or semantic vector search

Short-Term Memory: Checkpointers & State Compaction

The Checkpoint Engine

Every time a node finishes execution, the checkpointer serializes state and writes a snapshot tagged with `thread_id` and `checkpoint_id`.

Core Capabilities Unlocked

- **Time Travel & Rollbacks:** Inspect or resume execution from any earlier node state.
- **Human-in-the-Loop:** Pause graph execution for human approval, resuming seamlessly.

SNAPSHOT LIFECYCLE

Execution Steps: Node Ends → State Serialized → Snapshot Written.

Ensures fault tolerance and multi-turn state consistency across distributed graph nodes.

Context Bloat Solutions

High volume tool calls inflate thread history. Three core techniques maintain high model attention and fit token limits:

1. Sliding Window Trimming (`trim_messages`)

Truncates older messages based on token count while preserving system prompts and matching human/assistant message pairs.

2. Summarization Nodes

Conditional edge triggers when `len(messages) > THRESHOLD`, routing to an LLM that condenses turns into a summary block.

3. Tool Stripping

Replaces or drops verbose raw JSON/HTML `ToolMessage` payloads from thread state once synthesized by downstream agents.

Multi-Agent System Topologies



Hierarchical

Central supervisor delegates tasks to specialists.



Sequential

Assembly-line handoffs across pipeline stages.



Peer Swarm

Decentralized peer handoffs without coordinator.

Multi-Agent Collaboration & State Management

1. State Sharing: Global vs. Private

SHARED GLOBAL STATE (Graph Schema)

- **Canonical Thread:** Shared chat history & summaries passed between orchestrator and workers.
- **Workflow Control Flags:** Current phase, iteration counters, and error flags.
- **Shared Artifacts:** Final deliverables like structured specs, parsed code, or SQL schema.
- **Next Node Routing:** Routing key (`next_agent`) for conditional edges.

PRIVATE LOCAL STATE (Node Scoped)

- **Raw Tool Call Payloads:** Heavy HTML scrapes or 5K-row SQL dumps stay local; extract summaries only.
- **Node System Prompts:** Persona instructions kept inside function frame to prevent role mixing.
- **Internal Scratchpads:** Trial-and-error reasoning resolved locally before committing.
- **Sub-graph Frames:** Nested internal state stays isolated from parent state.

2. LangGraph Interaction Topologies

Supervisor / Router Pattern

Best for: Ambiguous queries & dynamic delegation

Central supervisor node inspects global state, dynamically selects worker agents via conditional edges, and evaluates results iteratively.

Network / Peer-to-Peer

Best for: Strict sequential handoffs (e.g. Triage → Billing)

Autonomous agents hand off execution directly to one another using node-level decisions via conditional graph edges without a supervisor.

Hierarchical / Sub-graphs

Best for: Complex enterprise multi-team workflows

Top-level supervisor delegates tasks to specialized sub-graphs, each running their own internal supervisor, worker nodes, and isolated states.

State Sharing & Context Isolation



Shared Global State

Persisted across the coordinator graph.

Task Status: High-level lifecycle completion flags.

Final Artifacts: Verified code and deliverables.

Handoff Payloads: Explicit messages between sub-agents.

Reducer Contracts: Schema-enforced state updates.



Private Worker State

Isolated within individual sub-agent nodes.

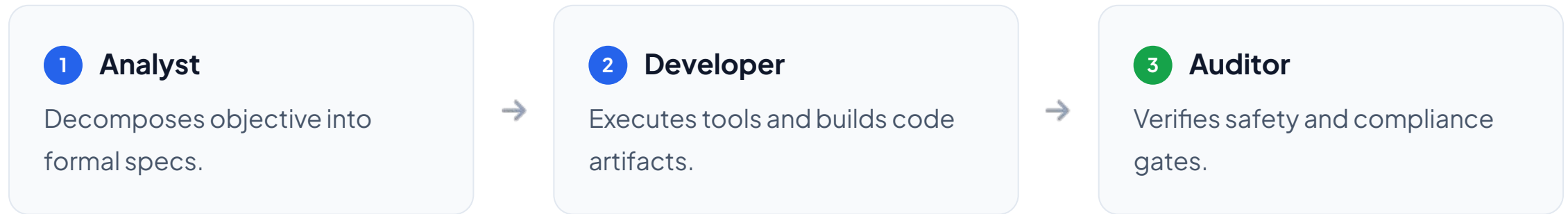
Scratchpad Logs: Raw reasoning and trial-and-error thoughts.

Raw Tool Dumps: Bulky API and shell traces.

System Prompts: Role-specific instructions and constraints.

Local Keys: Scoped least-privilege tool credentials.

Multi-Agent Pipeline with Verification



Failure Modes & Defensive Guardrails

Primary Vulnerabilities

Infinite Loops: Failed exit condition retries

Tool Hallucination: Invalid arguments

Prompt Injection: Poisoned tool inputs

Context Bloat: Observation memory saturation

Safety Precautions

Step Budgets: Hard limits on graph iterations

Pydantic Gates: Schema-enforced decoders

Least Privilege: Read-only default permissions

Pruning Nodes: Compacting historical state

Telemetry & Trajectory Benchmarks



PRODUCTION READINESS

Architecture Dictates Reliability

Autonomous capability scales only when backed by cyclic state graphs, explicit memory boundaries, and strict evaluation traces.

State Graphs • Checkpointing • Observability

References



https://cdn.prod.website-files.com/65d609edcc331dd0e4eb519b/683e5357b029e44c78368470_AD_4nXfxlXT3frqYTWxta-cGGwr-MeeOYAneTUrkViqRVyOJIQQphBEdNM1k9jMbq1KZhMpwBXdsGz3Kz3_vwSycetBe5Al8hQgFluk9Qsl-RQLqgwk2zOR0RZWWT38T16YSzfR_6LGUgA.png

Source: www.puppygraph.com



<https://www.youtube.com/@AIForYourWork>

Source: AI For Your Work (Slide Structure Inspiration)

Thank You!